



Manual de Usuario

Metro Subterráneo

Taller II

Integrantes:

Miguel Angel Santizo Zepeda – 2022021 ✓

Domingo Alejandro Romero Hernández - 2025157 ✓

Omar Alejandro Ottoniel Tobar de León - 2025092

Pablo Adrian Santos Monroy – 2023251 ✓

David Enrique Quintanilla Pérez-2022275

04/03/2026

Tecnologías Utilizadas

1. SQL (Structured Query Language)

Es el lenguaje estándar para comunicarte con bases de datos relacionales. Imagina que es el bibliotecario que sabe exactamente en qué estante y fila está la información que buscas.

- Para qué sirve: Crear, leer, actualizar y borrar datos (operaciones CRUD).
- Motores populares: PostgreSQL, MySQL, SQL Server y SQLite.
- Concepto clave: Los datos se organizan en tablas con filas y columnas, relacionadas entre sí mediante llaves (keys).

2. Git (Control de Versiones)

Git no es un lenguaje, sino una herramienta que rastrea los cambios en tu código. Es tu "máquina del tiempo".

- Para qué sirve: Evita que pierdas trabajo si algo sale mal y permite que varias personas trabajen en el mismo proyecto sin borrarse el código mutuamente.
- Comandos básicos: `git add`, `git commit`, `git push` y `git pull`.
- Plataformas: GitHub, GitLab y Bitbucket (son los "estantes" en la nube donde guardas tus repositorios de Git).

3. Postman (Pruebas y Desarrollo de APIs)

Si SQL es el bibliotecario y Git es la máquina del tiempo, Postman es el mensajero y el laboratorio de pruebas.

- Para qué sirve: Permite enviar peticiones (GET, POST, PUT, DELETE) a un servidor para ver qué responde, sin necesidad de tener lista la interfaz de usuario (el frontend).

- Uso principal: Probar que las APIs funcionen correctamente, documentar cómo deben usarse y automatizar pruebas de respuesta.
- Concepto clave: Trabaja con formatos de datos como JSON, que es el lenguaje universal para que el servidor y el navegador se entiendan.

1. ENTIDAD: PERSONAL

Esta entidad gestiona a los trabajadores asignados a las obras del metro.

A. Estructura de la Base de Datos

Campos:

id_personal (INT): Identificador único autoincremental.

nombre (VARCHAR 100): Nombre completo.

cargo (VARCHAR 50): Puesto administrativo u operativo.

B. Estructura en IntelliJ (Java)

Entity: Personal.java (Mapeo de tabla).

Repository: PersonalRepository.java (Consultas JPA).

Service: PersonalServiceImplements.java (Lógica de guardado y actualización).

Controles: PersonalController.java (Puntos de acceso API).

C. Uso en Postman

-Listar (GET): <http://localhost:8080/api/personal>

-Crear (POST): No requiere enviar el ID (es automático).

-Body (JSON):

```
{  
  "nombre": "Nombre del Empleado",  
  "cargo": "Cargo del Empleado"  
}
```

-Actualizar (PUT): Es obligatorio incluir el id_personal

D. Posibles Errores

ID Faltante: Al actualizar si envías el JSON sin el campo id_personal, el sistema devolverá un error 400 (Bad Request) indicando que es obligatorio.

2. ENTIDAD: IMPACTO DE TRÁFICO

Esta entidad mide la eficiencia del transporte y la reducción de tráfico por zonas.

A. Estructura de la Base de Datos

Campos:

id_impacto (INT): Identificador autoincremental.

zona (VARCHAR 100): Área de medición.

reduccion_trafico_porcentaje (DECIMAL 5,2): Valor porcentual.

B. Estructura en IntelliJ (Java)

-Entity: ImpactoTrafico.java (Incluye @GeneratedValue para el ID).

-Service: ImpactoServiceImplements.java (Valida que el porcentaje sea coherente).

-Controller: ImpactoController.java (Gestiona las peticiones HTTP).

C. Uso en Postman

-Listar (GET): <http://localhost:8080/api/impacto-trafico>

-Crear (POST): No requiere enviar el ID (es automático).

-Body (JSON):

```
{  
  "zona": "Zona Central",  
  "reduccionTraficoPorcentaje": 15.50  
}
```

-Actualizar (PUT): Es obligatorio incluir el id_impacto.

D. Posibles Errores

-Porcentaje Inválido: El sistema arrojará un error 400 si el valor es menor a 1 o mayor a 100 (Regla de negocio en el Controller).

-ID no encontrado: Intentar actualizar un ID que no existe en la base de datos no surtirá efecto o podría crear uno nuevo según la configuración.

3. ENTIDAD: HORARIOS

Esta entidad gestiona la programación de las salidas y llegadas de los trenes en el sistema de metro.

A. Estructura de la Base de Datos

Campos:

`id_horario` (INT): Id auto incrementable (Llave Primaria).

`hora_salida` (TIME): Hora programada de salida.

`hora_llegada` (TIME): Hora estimada de llegada.

`id_tren` (INT): Id del tren asignado (Llave Foránea).

B. Estructura en IntelliJ (Java)

Entity: `Horario.java` (Incluye `@GeneratedValue` para manejo automático de IDs).

Repository: `HorarioRepository.java` (Extiende de `JpaRepository`).

Service: `HorarioServiceImplements.java` (Implementa la lógica de negocio y guardado).

Controller: `HorarioController.java` (Puntos de acceso REST con la ruta `/api/horarios`).

C. Uso en Postman

Listar (GET): <http://localhost:8080/api/horarios>

Crear (POST): No requiere enviar el `idHorario` (el sistema lo genera).

Body (JSON):

```
1  {}  
2  |  
3  |  
4  |  
5  {}  
    "horaLlegada": "12:45:00",  
    "horaSalida": "15:00:00",  
    "idTren": 2
```

Actualizar (PUT): Es obligatorio incluir el `idHorario` en el cuerpo del JSON para identificar qué registro modificar.

D. Posibles Errores

Error 400 (Bad Request): Ocurre si el campo `idTren` es enviado como un número negativo o si las horas son nulas, debido a las anotaciones `@NotNull` y `@Positive` en la entidad.

Error 404 (Not Found): Se devuelve al intentar actualizar o eliminar un ID que no existe en la base de datos.

4. ENTIDAD: MANTENIMIENTO

Esta entidad registra todas las actividades técnicas, reparaciones y revisiones preventivas realizadas a los trenes.

A. Estructura de la Base de Datos

Campos:

`id_mantenimiento` (INT): Id auto incrementable.

`fecha` (DATE): Fecha en la que se realizó el mantenimiento.

`descripcion` (TEXT): Detalles del trabajo que se ejecuto.

`id_tren` (INT): ID del tren que recibió el servicio.

B. Estructura en IntelliJ (Java)

Entity: `Mantenimiento.java` (Mapeo de la tabla con soporte para tipos TEXT).

Repository: `MantenimientoRepository.java` (Consultas estándar de persistencia).

Service: `MantenimientoServiceImplements.java` (Gestiona la persistencia de los reportes).

Controller: `MantenimientoController.java` (Ruta base: `/api/mantenimiento`).

C. Uso en Postman

- Listar (GET): <http://localhost:8080/api/mantenimiento>

- Crear (POST): El `idMantenimiento` se genera automáticamente por la base de datos.

Body (JSON):

```
{
  "descripcion": "Actualización de software",
  "fecha": "2027-02-10",
  "idMantenimiento": 11,
  "idTren": 1
}
```

- Eliminar (DELETE): Se utiliza el ID en la URL (Ej: `/api/mantenimiento/1`).

D. Posibles Errores

Descripción Vacía: Si el campo `descripcion` se envía vacío o solo con espacios, el sistema devolverá un error 400 por la validación `@NotBlank`.

Formato de Fecha: Enviar la fecha en un formato distinto a `YYYY-MM-DD` causará un error de parseo en la API.

5. ENTIDAD: TREN

Esta entidad gestiona la información de los trenes que operan en el sistema de metro.

A. Estructura de la Base de Datos

Tabla: trenes

Campos:

id_tren **int** Identificador único autoincremental

modelo **varchar 50** Modelo del tren

capacidad_pasajeros **int** Capacidad máxima de pasajeros

estado **varchar 30** Estado actual del tren

```
CREATE TABLE trenes (  
-id_tren          int auto_increment primary key,  
-modelo          varchar(50) not null,  
-capacidad_pasajeros int not null,  
-estado          varchar(30) not null  
)
```

B. Estructura en IntelliJ (Java)

- Entity: Tren.java
- Repository: TrenRepository.java
- Service: TrenServiceImplements.java
- Controller: TrenController.java

C. Uso en Postman

Listar (GET)

<http://localhost:8080/api/trenes>

Crear (POST)

- No requiere enviar el idTren (el sistema lo genera automáticamente).

Actualizar (PUT)

- Es obligatorio incluir el idTren en el cuerpo del JSON para identificar qué registro modificar

Eliminar (DELETE)

<http://localhost:8080/api/trenes/{id}>

D. Posibles Errores

- Error 400 (Bad Request):
 - Ocurre si modelo o estado son nulos.
 - Si capacidadPasajeros es menor o igual a 0 (en caso de usar @Positive).
- Error 404 (Not Found):
 - Se devuelve al intentar actualizar o eliminar un ID que no existe en la base de datos.

6. ENTIDAD: CONDUCTOR

Esta entidad gestiona la información de los conductores asignados a los trenes del sistema de metro.

A. Estructura de la Base de Datos

Tabla: conductores

Campos: create table conductores (

id_conductor int auto_increment primary key,

nombre varchar(100) not null,

licencia varchar(50) not null unique,
telefono varchar(20),
estado varchar(30) not null
);

B. Estructura en IntelliJ (Java)

- Entity: Conductor.java
- Repository: ConductorRepository.java
- Service: ConductorServiceImplements.java
- Controller: ConductorController.java

C. Uso en Postman

Listar (GET)

<http://localhost:8080/api/conductores>

Crear (POST)

- No requiere enviar el idConductor (el sistema lo genera automáticamente).

Actualizar (PUT)

- Es obligatorio incluir el idConductor en el cuerpo del JSON.

Eliminar (DELETE)

<http://localhost:8080/api/conductores/{id}>

D. Posibles Errores

- Error 400 (Bad Request):
 - Si nombre, licencia o estado son nulos.
 - Si licencia ya existe (violación de restricción UNIQUE).
 - Si se usan validaciones como @Size o @NotBlank y no se cumplen.
- Error 404 (Not Found):
 - Al intentar actualizar o eliminar un conductor que no existe.

7. ENTIDAD: Línea

Esta entidad representa las rutas del tren dentro del sistema del metro

A. Estructura de la base de datos

Campos:

- Id_linea (int)
- Nombre_linea(varchar 50)
- Color (varchar 30)
- Longitud_km(decimal)

B. Estructura en IntelliJ

Entity: Linea:

- Mapeo de la tabla en la base de datos
- Incluye anotaciones como @Entity, @Id y @GeneratedValue
- Contiene las validaciones con @NotBlank, @Size, @Positive

Repository: LineaRepository:

- Extiende JpaRepository
- Permite realizar consultas automáticas con Jpa

Service: LineaServiceImplement:

- Contiene la lógica para guardar, actualizar, listar y eliminar líneas

Controller:LineaController:

- Gestiona los endpoints de la API (GET, POST, PUT, DELETE)



C. Uso en Postman

Listar (GET)

<http://localhost:8080/api/linea>

Crear (POST)

No requiere enviar el ID (es automático)

Body (JSON)

```
{  
  "nombreLinea": "Línea Roja",  
  "color": "Rojo",  
  "longitudKm": 25.50  
}
```

Actualizar (PUT)

Es obligatorio incluir el id_linea

```
{  
  "idLinea": 1,  
  "nombreLinea": "Línea Azul",  
  "color": "Azul",  
  "longitudKm": 30.00  
}
```

D. Validaciones

Las reglas de validación aplicadas son:

- **nombre_linea**
Obligatorio.
Debe tener entre 3 y 50 caracteres.
- **color**
Obligatorio.
Debe tener entre 3 y 30 caracteres.
- **longitud_km**
Obligatoria.
Debe ser un número positivo (mayor que 0).

E. Posibles Errores

- **Nombre o color muy cortos o vacíos:**
El sistema devolverá error **400 (Bad Request)** indicando que no cumple con el tamaño mínimo requerido.
- **Longitud negativa o igual a 0:**
Se devolverá error **400 (Bad Request)** porque la longitud debe ser positiva.
- **ID faltante en actualización:**
Si no se envía el ID en el método PUT, el sistema devolverá error 400.
- **ID no encontrado:**
Si el ID no existe en la base de datos, puede generar error 404 o no realizar ningún cambio según la configuración del Service.

8. ENTIDAD: Estación

Esta entidad representa las estaciones del metro, las cuales pertenecen a una zona específica

A.Estructura de la base de datos

Campos:

- **id_estacion (INT):** Identificador único autoincremental.
- **nombre (VARCHAR 100):** Nombre oficial de la estación.
- **zona (VARCHAR 100):** Nombre descriptivo de la zona donde se ubica.
- **id_zona (INT):** Llave foránea que hace referencia a la tabla Zona.

B.Estructura en IntelliJ

Entity: Estacion:

- Mapea la tabla en la base de datos.
- Incluye `@Entity`, `@Id`, `@GeneratedValue`.
- Relación con Zona usando `@ManyToOne` y `@JoinColumn(name="id_zona")`.

Repository: EstacionRepository

- Extiende `JpaRepository`.
- Permite realizar operaciones CRUD automáticas.

Service: EstacionServiceImplements

- Contiene la lógica para guardar, actualizar y eliminar estaciones.

Controller: EstacionController

Gestiona las peticiones HTTP (GET, POST, PUT, DELETE).

C. Uso en postman

Listar (GET)

<http://localhost:8080/api/estacion>

Crear (Post)

No requiere enviar el ID (es automático)

```
{  
  
  "nombre": "Estación Central",  
  
  "zona": "Zona Norte",  
  
  "idZona": 2  
  
}
```

Actualizar (PUT)

Es obligatorio incluir el id_estacion

```
{  
  
  "idEstacion": 1,  
  
  "nombre": "Estación Reforma",  
  
  "zona": "Zona Sur",
```


"idZona": 3

}

D. Uso en postman

Nombre

- Obligatorio
- No debe estar vacío

Zona

- Obligatorio
- No debe estar vacío

Id_zona

- Obligatorio
- Debe existir en la tabla Zona (Llave foránea válida)

E. Uso en postman

- Zona inexistente:

Si se envía un idZona que no existe, el sistema generará error 400 o error de clave foránea en la base de datos

- Campo vacío

El sistema devolverá error 400(Bad Request)

- ID faltante en actualización

Si no se envía id_estacion en el PUT, devolverá error 400

9. ENTIDAD: Pasajeros

Esta entidad muestra a los pasajeros.

A.Estructura de la base de datos

Campos:

- **id_pasajero(Int):** Identificador único autoincremental.
- **nombre (Varchar 50):** Nombre oficial de el pasajero.
- **tipo_pasajero(Varchar 30):** Descripción del tipo de pasajero.

B.Estructura en IntelliJ

Entity: Pasajero :

- Mapea la tabla en la base de datos.
- Incluye @Entity, @Id, @GeneratedValue.

Repository: PasajeroRepository

- Extiende JpaRepository.
- Permite realizar operaciones CRUD automáticas.

Service: PasajeroServiceImplements

- Contiene la lógica para guardar, actualizar y eliminar pasajeros.

Controller: PasajerosController

Gestiona las peticiones HTTP (GET, POST, PUT, DELETE).

C. Uso en postman

Listar (GET)



<http://localhost:8080/api/pasajeros>

Crear (Post)

No requiere enviar el ID (es automático)

```
{  
  
  "nombre": "Esdaurdo Ramirez",  
  
  "Tipo de pasajero": "Adulto",  
  
}
```

Actualizar (PUT)

Es obligatorio incluir el id_pasajero

```
{  
  
  "idPasajero": 1,  
  
  "nombre": "Estuardo Ramirez",  
  
  "Tipo de pasajero": "Adulto",  
  
}
```

D. Uso en postman

Nombre

- Obligatorio
- No debe estar vacío

Tipo

- Obligatorio
- No debe estar vacío

Id_pasajero

- Obligatorio
- Debe existir en la tabla Pasajero (Llave foránea válida)

E. Uso en postman

- Pasajero inexistente:

Si se envía un `idPasajero` que no existe, el sistema generará error 400 o error de clave foránea en la base de datos.

- Campo vacío

El sistema devolverá error 400.

- ID faltante en actualización

Si no se envía `id_pasajero` en el PUT, devolverá error 400

10. ENTIDAD: Boletos

Esta entidad muestra los boletos de los pasajeros.

A.Estructura de la base de datos

Campos:

- **id_boleto(Int):** Identificador único autoincremental.
- **precio(Decimal 10 , 2):** Precio del boleto.
- **fecha(Date):** Fecha de uso de el boleto.
- **id_pasajero(Int):** Llave foránea que hace referencia a la tabla Pasajeros .

B.Estructura en IntelliJ

Entity: Boleto :

- Mapea la tabla en la base de datos.
- Incluye @Entity, @Id, @GeneratedValue.
- Relación con Pasajeros usando @ManyToOne y @JoinColumn(name="id_pasajero")

Repository: BoletoRepository

- Extiende JpaRepository.
- Permite realizar operaciones CRUD automáticas.

Service: BoletoServiceImplements

- Contiene la lógica para guardar, actualizar y eliminar pasajeros.

Controller: BoletosController

Gestiona las peticiones HTTP (GET, POST, PUT, DELETE).

C. Uso en postman

Listar (GET)

<http://localhost:8080/api/boletos>

Crear (Post)

No requiere enviar el ID (es automático), solo se puede crear luego de crear al pasajero.

```
{  
  "precio": 150.00,  
  
  "fecha": "2026-03-03",  
  
  "pasajero": {  
  
    "idPasajero": 6  
  }  
}
```

Actualizar (PUT)

Es obligatorio incluir el id_boleto

```
{  
  
  "precio": 250.00,  
  
  "fecha": "2026-03-10",  
  
  "pasajero": {  
  
    "idPasajero": 5  
  }  
}
```

D. Uso en postman

Nombre

- Obligatorio
- No debe estar vacío

Tipo

- Obligatorio
- No debe estar vacío

Id_pasajero

- Obligatorio
- Debe existir en la tabla Pasajero (Llave foránea válida)

E. Uso en postman

- Pasajero inexistente:

Si se envía un `idPasajero` que no existe, el sistema generará error 400 o error de clave foránea en la base de datos.

- Campo vacío

El sistema devolverá error 400.

- ID faltante en actualización

Si no se envía `id_pasajero` en el PUT, devolverá error 400